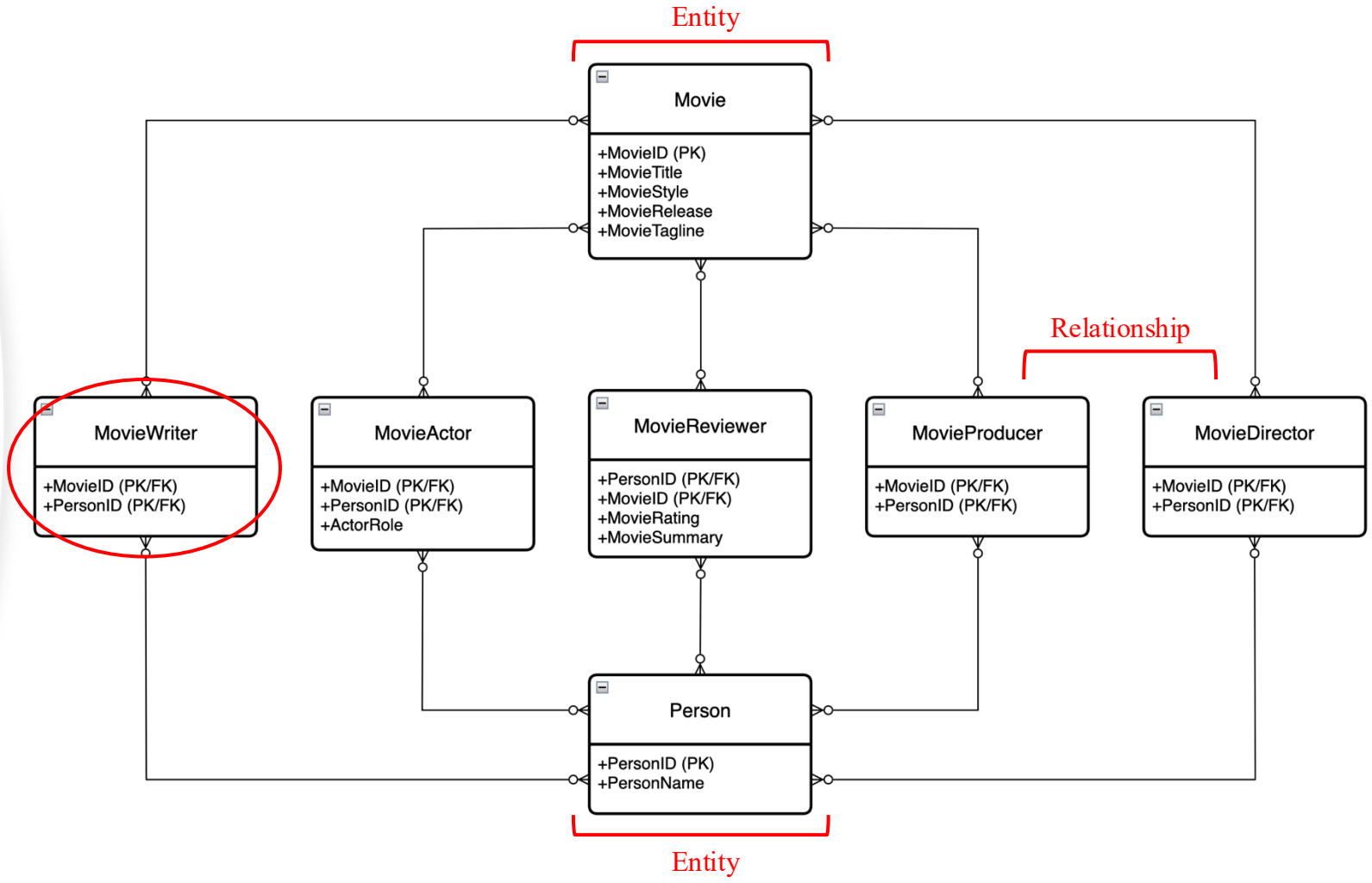
https://github.com/christianjhardin/CS-4630/tree/main

Graph to Relational Model Translation Project

Presented by Christian Hardin

Relational Database Schema of Movie Data

- All Relationships are between persons and movies as M:N:
  - For instance, a movie can have many actors, and an actor can act in many movies.
- Purpose of Junction tables:
  - Used when no single column is enough to uniquely identify a row (used for M:N).
  - For instance, to identify an actor's role in a movie, you need to pass along the the movie + actor as a single primary key.



Find all actors that directed a movie they also acted in and return actor and movie nodes

Returning the PersonName and MovieTitle

Using MovieActor (ma) and MovieDirector (md) we find where MovieID is the same in ma and md, AND where PersonID is the same in ma and md

Connecting PersonID in ma to PersonID in table Person (p) to get their name

Connecting MovieID in ma to table Movie (m) to get the title

```
SELECT
    p.PersonName,
    m.MovieTitle
FROM MovieActor ma
JOIN MovieDirector md
    ON ma.MovieID = md.MovieID
   AND ma.PersonID = md.PersonID
JOIN Person p
    ON ma.PersonID = p.PersonID
JOIN Movie m
    ON ma.MovieID = m.MovieID;
```

- Result:
  - Produces two columns displaying PersonName then MovieTitle.
- Runtime:
  - Would be very fast (a couple of seconds)
  - Results would be small—not many actors also direct
  - Very common SQL problem, so SQL handles it well.

Find all reviewer pairs, one following the other, and return the two reviewers and a movie they may have reviewed both

Returning both PersonName values AS Reviewer1 and Reveiwer2 to minimize confusion, and returning MovieTitle

Joining MovieReviewer twice to form a pair of reviewers and collecting those whose MovieID are the same.

Connecting PersonID in r1 to PersonID in table Person (p1) to get their name

Connecting PersonID in r2 to PersonID in table Person (p2) to get their name

Connecting MovieID in r1 to MovieID inf table Movie (m) to get movie title

```
SELECT
    p1.PersonName AS Reviewer1,
    p2.PersonName AS Reviewer2,
    m.MovieTitle
FROM MovieReviewer r1
JOIN MovieReviewer r2
    ON r1.MovieID = r2.MovieID
JOIN Person p1
    ON r1.PersonID = p1.PersonID
JOIN Person p2
    ON r2.PersonID = p2.PersonID
JOIN Movie m
    ON r1.MovieID = m.MovieID;
```

- Result:
  - Produces three columns displaying both reviewers' PersonName value, then MovieTitle.
- Runtime:
  - Fast for this dataset (<5 seconds)
  - Results compound as dataset size increases
  - Using n(n-1)/2:
    - 10 reviewers = 45 rows
    - 100 reviewers = 4,950 rows

Find all actor pairs that acted in multiple movies together

Returning both PersonName values AS Actor1 and Actor2 to minimize confusion, and counting the number of pairings in column MoviesTogether

Joining MovieActor twice to form a pair of actors and collecting those whose MovieID are the same.

Connecting PersonID in a1 to PersonID in table Person (p1) to get their name

Connecting PersonID in a2 to PersonID in table Person (p2) to get their name

Grouping by PersonID a1 then a2 to combine duplicate rows, and filtering out those pairing which have only one movie together

```
SELECT
    p1.PersonName AS Actor1,
    p2.PersonName AS Actor2,
    COUNT(*) AS MoviesTogether
FROM MovieActor a1
JOIN MovieActor a2
    ON a1.MovieID = a2.MovieID
JOIN Person p1
    ON a1.PersonID = p1.PersonID
JOIN Person p2
    ON a2.PersonID = p2.PersonID
GROUP BY
    a1.PersonID,
    a2.PersonID
HAVING COUNT(*) > 1;
```

- Result:
  - Produces three columns displaying both actors' PersonName value, then MoviesTogether value.
- Runtime:
  - Moderate for this dataset
  - (<8 seconds)
  - Results would usually be small, only selects if actors co-act multiple times
  - Many functions needing to operate in steps

Count the number of nodes reachable in at most 4 hops starting from Clint Eastwood ignoring edge direction

- Not possible with MySQL 8 and below because this problem requires non-native MySQL functions:
  - Recursion
  - Graph traversal
- Possible with MySQL 8+
  - MySQL 8+ includes the function: `WITH RECURSION`
  - Yet this problem would be extremely slow (30+ seconds) and complicated for MySQL to handle
  - Cypher can handle this question much easier given its database design
- Main difference between relational databases and graph databases for this question
  - In a relational database, only entities have attributes and relationships are implicit through FK's and joins.
  - In a graph database, both entities and relationships can have attributes and are explicit.
  - Cypher is like GPS, while SQL is like a collection of tables with street names needing to connect and reconnect every time a movement is needed.